

FSM and datapath in RTL

You have already met finite-state machines, ALU datapaths, and memory maps as digital concepts

Control plus datapath

- In RTL, the FSM usually lives in a clocked always block with a case on the current state
- Next-state logic and Moore outputs can share that block or split for clarity
- The datapath is mostly combinational
- A memory map in hardware is a register file or RAM block addressed by an index
- You do not need a perfect one-to-one copy of every lab UI

Revisit the concept labs

Digital Design and Verification Platform

HomeTools

Home / Tools / FSM lab

INTERACTIVE TOOL

FSM designer + stepper

Edit a small finite-state machine (Moore or Mealy), see the state table, then step a bit stream and watch state / output. Starter: 2-state Moore toggle — Z flips when input $x=1$.

Starter example: Moore toggle — S0 (Z=0) / S1 (Z=1); $x=1$ flips state. Step 1011 and watch Z.

Challenge 0 / 22 cleared

Quiz: FSM: A finite-state machine is defined by...

☐ a finite set of states, inputs, outputs, and next-state / output rules

☐ only a continuous assign

☐ an infinite tape

☐ a cache tag array

Show hint

Check

Next

Load challenge setup

Idle

Quiz: FSM

Quiz: Moore

Quiz: Mealy

Quiz: state table

Quiz: stepping

Quiz: reset run

Quiz: vs seq-detector

Quiz: encoding

Toggle starter

Toggle on 1

Hold on 0

Mealy pulse

Ring C

Edit next-state

Moore Z from state

FSM lab — Moore toggle starter

Real Verilog practice

```
wsl - module18-fsm-datapath-bridge/examples/navigation

# Track A - real Unix
# real Verilog session (Track A)

$ cat examples/fsm_bridge_demo/fsm_bridge_demo.v
// fsm_bridge_demo.v - Moore FSM and simple ALU
module moore_toggle(
    input    clk,
    input    rst,
    input    x,
    output reg z
);
    reg state;
    localparam S0 = 1'b0, S1 = 1'b1;

    always @(posedge clk) begin
        if (rst) state ≤ S0;
        else case (state)
            S0: state ≤ x ? S1 : S0;
            S1: state ≤ x ? S0 : S1;
        endcase
    end

    always @(posedge clk) begin
        if (rst) z ≤ 1'b0;
        else case (state)
            S0: z ≤ 1'b0;
            S1: z ≤ 1'b1;
        endcase
    end
endmodule

module simple_alu(
    input [3:0] a,
    input [3:0] b,
    input [1:0] op,
    output reg [3:0] y
);
    always @(*) begin
        case (op)
            2'b00: y = a + b;
            2'b01: y = a - b;
            2'b10: y = a & b;
            default: y = a ^ b;
        endcase
    end
endmodule

$ iverilog -t null examples/fsm_bridge_demo/fsm_bridge_demo.v
(syntax check passed)
```

Real Verilog - FSM and ALU sketches

Pitfalls to watch

- Keep clocked and combo roles clear
- Do not forget a default state after reset or synthesis may latch undefined behavior
- Do not assume the browser lab is the only spec
- And do not skip the memory-map lab entirely

Your turn

- Complete the checklist
- Pick one concept lab, load the starter, and outline a Verilog module with matching ports
- Write at least a module shell and one always or assign block
- Optionally run iverilog or the browser HDL simulator on your sketch
- When you finish, take the short quiz, then continue to the course wrap in the next module